

Ask, Retrieve, Act

Capstone Project — Technical Documentation

Version 1.0

Audience: Interns (post-MCP, post-RAG)

Purpose: Complete build reference — architecture, contracts, schemas, node-by-node agent design, setup, testing, deployment, and grading criteria.

Table of Contents

1. Project Summary
2. System Architecture
3. Repository Structure
4. Environment Setup
5. Data Layer — PostgreSQL / Prisma Schema
6. Knowledge Base — RAG Pipeline Spec
7. Tool Layer — MCP Server Spec
8. Agent Layer — LangGraph Graph Spec (node-by-node)
9. Backend API — FastAPI Contract
10. Frontend — Next.js Chat UI Spec
11. Caching Strategy — Redis
12. Observability & Audit Logging
13. Guardrails & Failure Handling
14. Testing Strategy
15. CI/CD Pipeline
16. Deployment Checklist
17. Evaluation Rubric (detailed)
18. Milestone-by-Milestone Checklist
19. Common Pitfalls
20. Glossary

1. Project Summary

Build an agent that answers questions from a private knowledge base and takes real actions through tools, deciding for itself — at runtime — whether a query needs retrieval, a tool call, both, or neither.

Non-negotiable requirements:

- A genuine planning step that conditions on the query (not hardcoded routing)
- At least one query that requires retrieval and a tool call chained together
- Graceful failure handling (no hallucinated answers when data is missing)
- Full audit trail (what was retrieved, what tools were called, with what inputs/outputs)
- One deliberate safety guardrail (e.g. confirm-before-destructive-action)

Everything in this document exists to make those five requirements concrete and buildable.

2. System Architecture

The system is a chat application backed by an agent graph, a knowledge base, and a tool layer. Below is the flow of a single request, described step by step in place of the original flowchart diagram.

Request flow

- 1. User sends a message from the Next.js chat UI.
- 2. FastAPI receives it, loads conversation history from Postgres, and invokes the LangGraph agent.
- 3. The Planner node decides the path: retrieval only, tool call only, both (chained), or 'answer directly / ask for clarification.'
- 4. The Retriever node queries the vector DB (checking Redis cache first) and returns top-k chunks with similarity scores.
- 5. The Tool-Executor node calls the MCP server's tools with structured arguments and gets structured results back.
- 6. The Responder node synthesizes the final answer from whatever context is available, citing the sources/tools used.
- 7. If confidence is low or information is missing, the Clarify node fires instead of a fabricated answer.
- 8. Every step (planner decision, retrieved chunks + scores, tool calls + I/O, final answer) is logged to Postgres for audit and to LangSmith / structured logs for tracing.

Component map:

Component	Role
Next.js Chat UI	Sends user messages, renders streamed responses and the step trace
FastAPI Backend	Orchestrates the request, talks to Postgres, invokes the agent graph
LangGraph Agent	Planner → Retriever / Tool-Executor → Responder → (Clarify)
Pinecone / Chroma	Vector store for the knowledge base
Redis	Caches retrieval results

Component	Role
MCP Server	Exposes tools (order lookup, ticket lookup, create ticket) to the agent
Postgres	Stores conversations, messages, and the full audit log
LangSmith / structured logs	Traces every run for debugging

3. Repository Structure

```

capstone/
├── apps/
│   ├── web/                                # Next.js frontend
│   │   ├── app/
│   │   │   ├── chat/page.tsx
│   │   │   └── api/proxy/                # optional proxy to backend if same-origin needed
│   │   ├── components/
│   │   │   ├── ChatWindow.tsx
│   │   │   ├── MessageBubble.tsx
│   │   │   └── AgentStepTrace.tsx        # shows which node/tool fired, for transparency
│   │   └── lib/api.ts
│   └── api/                                # FastAPI backend
│       ├── main.py
│       ├── routes/
│       │   ├── chat.py
│       │   ├── health.py
│       │   └── admin.py                  # reindex endpoint
│       ├── agent/
│       │   ├── graph.py                  # LangGraph definition
│       │   ├── nodes/
│       │   │   ├── planner.py
│       │   │   ├── retriever.py
│       │   │   ├── tool_executor.py
│       │   │   ├── responder.py
│       │   │   └── clarify.py
│       │   └── state.py                  # shared graph state schema
│       ├── rag/
│       │   ├── ingest.py
│       │   ├── embed.py
│       │   └── retrieve.py
│       ├── mcp_client/
│       │   └── client.py                 # connects to the MCP server(s)
│       ├── db/
│       │   ├── prisma/schema.prisma
│       │   └── session.py
│       ├── cache/redis_client.py
│       └── tests/
├── mcp-server/                             # extended from their earlier MCP task
│   ├── server.py
│   ├── tools/
│   │   ├── ticket_lookup.py
│   │   ├── order_lookup.py
│   │   └── create_ticket.py             # the 'destructive' action needing confirmation
│   └── mock_data/
├── docs/
│   ├── architecture.md
│   └── eval-set.json                    # test queries for the eval harness (stretch goal)

```

```
|  
├── .github/workflows/ci.yml  
├── docker-compose.yml      # postgres + redis for local dev  
└── README.md
```

4. Environment Setup

Prerequisites: Node 18+, Python 3.11+, Docker (for Postgres/Redis locally), a Pinecone API key (or swap in Chroma/FAISS for a no-cost local vector store), and an LLM API key.

```
# 1. Clone and enter  
git clone <repo-url> && cd capstone  
  
# 2. Local infra  
docker compose up -d  # starts postgres:5432 and redis:6379  
  
# 3. Backend  
cd apps/api  
python -m venv .venv && source .venv/bin/activate  
pip install -r requirements.txt  
cp .env.example .env  # fill in DB_URL, REDIS_URL, PINECONE_API_KEY, LLM_API_KEY  
prisma generate  
prisma migrate dev --name init  
uvicorn main:app --reload --port 8000  
  
# 4. MCP server (separate process)  
cd ../../mcp-server  
pip install -r requirements.txt  
python server.py  # runs on its configured stdio/http transport  
  
# 5. Frontend  
cd ../apps/web  
npm install  
cp .env.local.example .env.local  # set NEXT_PUBLIC_API_URL=http://localhost:8000  
npm run dev
```

Suggested .env keys (backend):

```
DATABASE_URL=postgresql://user:pass@localhost:5432/capstone  
REDIS_URL=redis://localhost:6379  
PINECONE_API_KEY=...  
PINECONE_INDEX=capstone-kb  
LLM_API_KEY=...  
LLM_MODEL=<the model you're using>  
MCP_SERVER_URL=http://localhost:9000  # or stdio config if using stdio transport  
LANGCHAIN_TRACING_V2=true             # if using LangSmith  
LANGCHAIN_API_KEY=...
```

5. Data Layer — PostgreSQL / Prisma Schema

```
// apps/api/db/prisma/schema.prisma  
  
model Conversation {  
  id          String      @id @default(uuid())  
  createdAt   DateTime     @default(now())  
  messages    Message[]  
}
```

```

}

model Message {
  id          String      @id @default(uuid())
  conversationId String
  conversation Conversation @relation(fields: [conversationId], references: [id])
  role        String      // "user" | "assistant"
  content     String
  createdAt   DateTime     @default(now())
  auditLogs   AuditLog[]
}

model AuditLog {
  id          String      @id @default(uuid())
  messageId   String
  message     Message     @relation(fields: [messageId], references: [id])
  stepType    String      // "plan" | "retrieval" | "tool_call" | "clarify" | "final_answer"
  stepDetail   Json       // structured payload — see shapes below
  createdAt   DateTime @default(now())
}

```

stepDetail JSON shapes (document these — graders will check them):

```

// stepType = "plan"
{ "decision": "retrieve_and_tool_call", "reasoning": "user asked about policy AND a specific order" }

// stepType = "retrieval"
{ "query": "refund policy", "chunks": [
  { "docId": "policy-042", "text": "...", "score": 0.83 },
  { "docId": "policy-011", "text": "...", "score": 0.79 }
] }

// stepType = "tool_call"
{ "tool": "order_lookup", "input": { "order_id": "4521" }, "output": { "status": "eligible", "days_since_purchase": 12 } }

// stepType = "clarify"
{ "reason": "no matching order found", "question_asked": "Could you double check the order number?" }

// stepType = "final_answer"
{ "sources_used": ["policy-042"], "tools_used": ["order_lookup"] }

```

6. Knowledge Base — RAG Pipeline Spec

Document set: 20–50 documents minimum. Suggested sources: fictional company policy docs, FAQ pages, product docs, or real open-source project docs if you'd rather not write fixtures.

Ingestion (rag/ingest.py)

- Load documents (markdown/txt/PDF — pick one format to keep it simple).
- Chunk: 300–500 tokens per chunk, ~15% overlap. Document why you chose this size in your README — graders will ask.
- Embed each chunk (any embedding model consistent with your LLM provider).
- Upsert into Pinecone with metadata: { docId, title, section, source_url? }.

Retrieval (rag/retrieve.py)

- Given a query, embed it, run top-k similarity search (k=4–6 to start).
- Cache first: check Redis for an identical or near-identical query before hitting Pinecone; cache the result with a TTL (e.g. 10 minutes).
- Return chunks with similarity scores — the planner and responder both need these to judge confidence.
- Minimum relevance threshold: if the top score is below a threshold you define (e.g. 0.65), treat retrieval as 'no relevant results' rather than forcing an answer from weak matches. This feeds the Clarify path.

7. Tool Layer — MCP Server Spec

Extend the MCP server from the earlier intern task. Minimum two tools, ideally three:

Tool	Input schema	Output schema	Notes
order_lookup	{ order_id: string }	{ status, eligible, days_since_purchase }	Mock or real backing data
ticket_lookup	{ ticket_id: string }	{ status, assignee, last_update }	
create_ticket	{ subject, description, priority }	{ ticket_id, created }	Destructive-ish action — guardrail candidate (Section 13)

Each tool must:

- Validate its input schema and return a structured error object on invalid input (not a raw exception) — this feeds failure handling.
- Be independently testable via a direct MCP call, outside the full agent graph.
- Log every invocation (input + output) — the tool-executor node persists this to AuditLog.

8. Agent Layer — LangGraph Graph Spec

Shared state (agent/state.py):

```
class AgentState(TypedDict):
    conversation_id: str
    user_query: str
    plan: Optional[dict]          # planner's decision + reasoning
    retrieved_chunks: Optional[list]
    tool_results: Optional[list]
    final_answer: Optional[str]
    needs_clarification: bool
    clarification_question: Optional[str]
```

Planner Node

- Input: user_query, recent conversation history.
- Job: classify the query into one of: retrieve_only, tool_only, retrieve_and_tool (chained), answer_directly (e.g. greetings/meta questions), or insufficient_info.

- This must be an LLM call reasoning over the query — not a keyword if/else. Graders will use an adversarial query specifically designed to break naive keyword routing to check this.
- Output: writes plan into state, including a short reasoning string (this is what gets audit-logged).

Retriever Node

- Runs only if plan.decision includes retrieval.
- Calls rag/retrieve.py, writes retrieved_chunks into state.
- If nothing clears the relevance threshold, sets a flag the Responder/Clarify nodes can read (don't silently proceed as if nothing happened).

Tool-Executor Node

- Runs only if plan.decision includes a tool call.
- Extracts structured arguments for the tool from the query (an LLM call producing the tool's input schema, or the planner extracts this up front — document your choice).
- Calls the MCP client, writes tool_results into state.
- Guardrail check happens here for destructive actions (see Section 13) before actually calling create_ticket.
- On tool failure (bad input, mock API error), writes a structured failure into tool_results rather than raising — the Responder decides how to phrase this to the user.

Responder Node

- Synthesizes retrieved_chunks + tool_results (whichever are populated) into a final answer.
- Must explicitly cite which doc(s)/tool(s) it used — this becomes the sources_used / tools_used audit entry.
- If retrieval came back empty/low-confidence and no tool data resolves the question, routes to Clarify instead of answering.

Clarify Node

- Produces a specific clarifying question (not a generic 'can you rephrase?').
- Writes needs_clarification = True and clarification_question.
- The API layer returns this as the assistant's turn; the next user message re-enters the graph with the added context.

Conditional edges:

Planner → (Retriever and/or Tool-Executor) → Responder → (END or Clarify → END). Draw this exact graph before writing code — it is the Milestone 1 deliverable.

9. Backend API — FastAPI Contract

POST /chat

Request:

```
{ "conversation_id": "uuid-or-null-for-new", "message": "What's our refund policy, and is order 4521 eligible?" }
```

Response (streamed, Server-Sent Events or chunked):

```
{ "type": "step", "step": "plan", "detail": { "decision": "retrieve_and_tool_call" } }
```

```
{ "type": "step", "step": "retrieval", "detail": { "chunks": [...] } }
{ "type": "step", "step": "tool_call", "detail": { "tool": "order_lookup", "output": {...} } }
{ "type": "final", "content": "Our refund policy allows... Order #4521 is eligible because...",
  "sources_used": ["policy-042"], "tools_used": ["order_lookup"] }
```

Streaming the intermediate step events is what powers the frontend's transparency panel (Section 10) — don't just stream the final tokens.

GET /health

Returns { "status": "ok", "db": "ok", "redis": "ok", "vector_store": "ok" } — check each dependency, don't just return 200 unconditionally.

POST /admin/reindex

Triggers rag/ingest.py against the current document set. Protect with a simple API key header — doesn't need real auth for this project.

10. Frontend — Next.js Chat UI Spec

- Chat window with streaming message rendering.
- AgentStepTrace component: below or beside each assistant message, show a collapsed trace of what happened — 'Retrieved 3 docs (top score 0.83) → Called order_lookup(order_id=4521) → Answered.' This is a graded requirement, not a nice-to-have.
- Clarifying questions render like a normal assistant message, but keep needs_clarification in local state so the UI can visually flag 'waiting on you' if you want to go further.
- No auth needed for the capstone unless you want to add it as a stretch.

11. Caching Strategy — Redis

- Cache key: hash of the normalized retrieval query string.
- Cache value: the retrieved chunks + scores (JSON).
- TTL: 5–15 minutes is reasonable for a demo; document your choice and why.
- Cache is for retrieval results only in v1 — do not cache tool call results (order/ticket data should stay live).

12. Observability & Audit Logging

- Every graph run must produce a full AuditLog trail in Postgres (see schema in Section 5) — this is graded directly.
- If you have LangSmith access, wrap the graph invocation so every run is traced there too (gives a visual timeline for debugging, separate from the Postgres audit trail which is the permanent record).
- If no LangSmith access: structured JSON logs to stdout with a trace_id per request, at minimum.

13. Guardrails & Failure Handling

Required guardrail (pick this one or design your own of equal weight):

create_ticket must not fire on the first pass. When the planner detects a request that maps to a destructive/creating action, the Responder returns a confirmation question ("I'll create a ticket titled '...' — confirm?") and only the tool-executor actually calls create_ticket after an explicit affirmative in the next turn.

Required failure handling behaviors:

- Retrieval returns nothing relevant → Clarify, don't answer from unrelated chunks.
- Tool call fails (bad ID, mock API down) → tell the user plainly what failed, don't fabricate a result.
- Ambiguous query (could mean two different tools/docs) → ask which one, don't guess silently.

14. Testing Strategy

- Unit tests: each node in isolation (mock the LLM call for the planner in fast tests; one integration test hits the real LLM).
- Tool tests: each MCP tool tested directly, independent of the agent graph.
- Integration test (required): the full multi-step query from Section 4's example ("refund policy + order 4521") run end-to-end, asserting both a retrieval step and a tool-call step appear in the audit log.
- Failure-path tests: one test for empty retrieval, one for tool failure, one for the guardrail confirmation flow.

15. CI/CD Pipeline

.github/workflows/ci.yml should, at minimum:

- 1. Install backend deps, run pytest.
- 2. Install frontend deps, run lint + build.
- 3. (Stretch) Run the eval harness (docs/eval-set.json) against a staging LLM call and fail the build if accuracy drops below a threshold you define.

16. Deployment Checklist

- ☐ Environment variables set in the deployment target (never commit .env)
- ☐ Postgres migrations run (prisma migrate deploy)
- ☐ Vector index populated (run /admin/reindex once post-deploy)
- ☐ Redis reachable from the deployed backend
- ☐ MCP server reachable from the deployed backend (same network or public URL)
- ☐ CORS configured on FastAPI for the deployed frontend origin
- ☐ Health check endpoint returns green before demoing

17. Evaluation Rubric (detailed)

Category	Pts	What graders check
Architecture correctness	25	Planner makes a real LLM-based decision on an adversarial test query; graph has real conditional branching, not scripted sequencing

Category	Pts	What graders check
RAG quality	20	Chunking rationale documented, relevant chunks retrieved for 5 sample queries, similarity scores surfaced, threshold logic works
Tool integration	20	Both/all MCP tools callable directly and through the agent; correct argument extraction; results correctly reflected in final answer
Robustness	15	Empty retrieval, tool failure, and ambiguous-query tests all pass; guardrail confirmation flow works exactly as specified
Code & observability	10	Clean module boundaries matching this doc's structure; full audit trail present in Postgres for every test run; tests pass in CI
Demo & documentation	10	Can explain graph shape decisions live; README documents chunking size, threshold values, and the chosen guardrail with reasoning

18. Milestone-by-Milestone Checklist

Week 1

- ☐ Graph diagram drawn and reviewed before any code
- ☐ FastAPI skeleton running, /health green
- ☐ Prisma schema migrated
- ☐ 20+ documents chunked and upserted into vector store

Week 2

- ☐ Planner, Retriever, Tool-Executor, Responder nodes implemented
- ☐ MCP client wired to at least 2 tools
- ☐ The chained retrieval+tool query works end-to-end once, manually verified

Week 3

- ☐ Frontend chat UI streaming responses
- ☐ AgentStepTrace shows real step data
- ☐ Redis caching live
- ☐ Guardrail + all three failure-path behaviors implemented
- ☐ Test suite covers Section 14 minimums

Week 4

- ☐ CI pipeline green
- ☐ Deployment checklist complete
- ☐ README finished (architecture, setup, chunking/threshold rationale, guardrail rationale)
- ☐ Demo rehearsed, including one deliberately broken query live

19. Common Pitfalls

- Hardcoding the planner as if "order" in query: ... — this fails the architecture-correctness grading criterion immediately. The decision must come from an LLM call reasoning over the query.
- Streaming only final tokens, not step events — the frontend transparency requirement needs the intermediate step events, not just the answer.
- Caching tool results — tool data (orders, tickets) must stay live; only cache retrieval.
- Answering with no relevant chunks anyway — if the top similarity score is below your documented threshold, that's a Clarify case, not an answer case.
- Skipping the guardrail confirmation round-trip — create_ticket firing on the first message is an automatic points loss in Robustness.

20. Glossary

MCP (Model Context Protocol): standardized way to expose tools/data sources to an LLM agent.

RAG (Retrieval-Augmented Generation): grounding LLM answers in retrieved documents rather than parametric memory alone.

Planner node: the graph node responsible for deciding what to do with a query before any retrieval/tool execution happens.

Guardrail: a deliberate constraint preventing the agent from taking an action without an explicit checkpoint (here: confirmation before a destructive/creating tool call).

Audit trail: the persisted, per-step record of what the agent retrieved, called, and decided — required for grading and for real-world debuggability.